

Phishing Website Detection using Machine Learning

A Critical Reproduction and Evaluation Study

Submitted by:

Dana Shadafne

Course:

Data Science in Cyber

Instructor:

Dr. Uri Itai

University of Haifa

Date:

July 2026

Table of Contents

1. Executive Summary	3
2. Summary of the Source.....	5
3. Critical Evaluation	7
4. Feature Engineering Analysis	10
5. Reproducibility Analysis	14
6. Experimental Results	16
7. Conclusions.....	22
References.....	23

1. Executive Summary

This project presents a critical reproduction and evaluation of the "Phishing Website Detection by Machine Learning Techniques" project. The goal of the original work is to detect phishing websites using machine learning models trained on URL and domain-based features.

Phishing attacks represent a major cybersecurity threat because attackers create fraudulent websites that imitate legitimate services in order to steal sensitive information such as login credentials, personal information, and financial data. Therefore, automated detection methods are needed to identify malicious websites efficiently.

The original project proposed using extracted website characteristics and machine learning classifiers to distinguish phishing websites from legitimate ones. In this reproduction study, the dataset and methodology were analyzed, and multiple classification models were implemented and compared, including Logistic Regression, Decision Tree, Random Forest, and XGBoost.

The experimental evaluation was extended beyond basic accuracy measurement by including additional metrics such as Precision, Recall, F1-score, ROC-AUC, Matthews Correlation Coefficient (MCC), and confusion matrix analysis. These metrics provide a more complete understanding of model performance in a cybersecurity context.

The results showed that ensemble-based models, especially Random Forest and XGBoost, achieved the strongest performance. Feature importance analysis indicated that URL-based characteristics such as URL length, URL depth, and prefix/suffix patterns were among the most influential indicators for detecting phishing behavior.

The critical evaluation showed that although the original approach demonstrates the effectiveness of machine learning for phishing detection, some limitations exist. The balanced dataset may not fully represent real-world conditions where phishing websites are much less frequent than legitimate websites. In addition, phishing techniques continuously evolve, requiring models to be updated and evaluated on newer datasets.

Overall, the reproduction supports the author's main conclusion that machine learning techniques can effectively assist in phishing website detection. However, additional real-world validation, updated datasets, and continuous model improvement are necessary before practical deployment.

2. Summary of the Source

The selected source for this project is "Phishing Website Detection by Machine Learning Techniques", which provides a machine learning based approach for identifying phishing websites using URL and domain-related features.

The original implementation, including the dataset and source code, is publicly available through the project's GitHub repository.

The main problem addressed by the project is the detection of phishing websites. Phishing attacks are a common cybersecurity threat where attackers create fraudulent websites that imitate legitimate services in order to steal sensitive information such as passwords, personal information, or financial credentials.

Traditional blacklist-based phishing detection approaches are limited because they mainly identify previously reported malicious websites and may fail to detect newly created phishing domains. Therefore, machine learning methods can provide a more adaptive solution by learning patterns that distinguish phishing websites from legitimate ones.

The proposed solution extracts different characteristics from website URLs and domain information and uses these features to classify websites as either legitimate or phishing. Instead of relying on manual rules, the system learns patterns from previously collected examples.

The dataset used in the original implementation contains 10,000 website samples divided into two classes: 5,000 legitimate websites and 5,000 phishing websites. Each website is represented using extracted features such as URL length, URL depth, use of special characters, domain information, redirection behavior, DNS records, and website traffic information.

The original project applied several machine learning techniques to evaluate whether URL-based features can effectively distinguish phishing websites from legitimate ones. The main idea is to train classification models on extracted features and evaluate their ability to generalize to unseen websites.

3. Critical Evaluation

The original project demonstrates that machine learning techniques can be successfully applied to phishing website detection using extracted URL and domain-based features. The main claim of the author is that machine learning classifiers are capable of distinguishing phishing websites from legitimate websites by learning patterns from historical data.

The reproduced experiments support the general claim that machine learning is effective for this task. The trained models achieved strong classification performance, showing that the selected features contain meaningful information for detecting phishing behavior.

However, although the results are promising, several methodological limitations exist that should be considered before applying this system in a real cybersecurity environment.

3.1 Evaluation Methodology

The original implementation mainly focuses on comparing machine learning models based on their prediction performance. However, phishing detection is a security-sensitive task where accuracy alone is not sufficient.

In cybersecurity applications, different types of classification errors have different consequences. A False Positive occurs when a legitimate website is incorrectly classified as phishing, which may inconvenience users. A False Negative occurs when a phishing website is classified as legitimate, allowing a malicious website to remain undetected.

False Negatives are usually more dangerous in phishing detection because they may expose users to credential theft, financial loss, or privacy risks. Therefore, evaluation metrics such as Recall, F1-score, ROC-AUC, and Matthews Correlation Coefficient (MCC) provide a more complete understanding of model behavior compared with accuracy alone.

For this reason, the reproduced experiments extended the evaluation by including multiple metrics and confusion matrix analysis.

3.2 Dataset Limitations

Although the dataset provides useful examples for training and evaluating phishing detection models, several limitations should be considered.

The dataset used in the project is perfectly balanced, containing an equal number of phishing and legitimate websites. While this is useful during model development, it may not accurately represent real-world cybersecurity scenarios. In reality, legitimate websites are significantly more common than phishing websites, creating a natural class imbalance problem.

Because of this difference, evaluation results obtained on a balanced dataset may overestimate the performance of the model in real deployment environments. A model that performs well on balanced data may generate different False Positive and False Negative rates when applied to real internet traffic.

Another limitation is that phishing techniques continuously evolve. Attackers frequently modify URLs, domain structures, and website behavior to avoid detection. Therefore, a model trained on historical phishing examples may lose effectiveness over time unless it is regularly updated with new data.

3.3 Evaluation of the Author's Conclusions

The author's conclusion that machine learning methods are effective for phishing website detection is generally supported by the experimental results. The reproduced experiments confirmed that URL-based and domain-based features can provide strong predictive information.

However, the original work does not fully address important real-world deployment challenges. Additional evaluation on newer datasets, highly imbalanced datasets, and real-time phishing examples would be necessary before concluding that the system is ready for practical cybersecurity use.

Furthermore, the original implementation provides limited discussion about model interpretability. Understanding why a model classifies a website as phishing is important in cybersecurity applications, where analysts need to trust and investigate automated decisions.

Overall, the original project provides a strong foundation and demonstrates the potential of machine learning for phishing detection, but further validation and improvement are required for deployment in real-world environments.

4. Feature Engineering Analysis

Feature engineering is an important step in phishing website detection because the model does not analyze the website directly. Instead, it learns from measurable characteristics extracted from URLs and domain information.

The original project performed feature extraction by converting website properties into numerical features that can be processed by machine learning algorithms. These features describe different characteristics that may indicate suspicious website behavior.

4.1 Features Used

The dataset contains multiple categories of features:

URL-based features:

These features analyze the structure of the website URL. Examples include URL_Length, URL_Depth, Prefix/Suffix usage, and the presence of special patterns. These features are useful because phishing websites often use unusually long URLs, multiple subdirectories, or misleading domain structures to imitate legitimate websites.

Domain-based features:

Features such as Domain_Age and DNS_Record provide information about the history and reliability of a domain. Newly created domains or domains with missing information may indicate suspicious behavior because phishing websites are often created for short-term attacks.

Behavior-based features:

Features such as Redirection, Web_Traffic, Mouse_Over, and Right_Click describe website behavior. These characteristics can help identify websites attempting to hide malicious actions or manipulate user interaction.

4.2 Preprocessing and Transformations

Before training the models, preprocessing steps were applied to prepare the dataset. The Domain column was removed because it contains textual identifiers rather than numerical information useful for the selected machine learning models.

Feature scaling was performed using StandardScaler, which transforms features by subtracting the mean and dividing by the standard deviation. Standardization ensures that numerical features have comparable scales, which is especially important for algorithms such as Logistic Regression that are sensitive to feature magnitude.

Tree-based models such as Decision Tree, Random Forest, and XGBoost are less affected by scaling because they split data based on feature thresholds. However, applying consistent preprocessing allows fair comparison between different algorithms.

4.3 Feature Importance Analysis

To better understand the contribution of each feature, feature importance analysis was performed using the Random Forest model. This analysis helps identify which characteristics have the strongest influence on the classification decision.

The results showed that URL-based features contributed the most to phishing detection. The most influential features were URL_Length, URL_Depth, and Prefix/Suffix.

URL_Length had the highest importance score. This is reasonable from a cybersecurity perspective because phishing websites frequently use long URLs containing additional words, parameters, or misleading structures to appear legitimate.

URL_Depth was also highly important because phishing URLs may contain multiple directory levels in an attempt to hide suspicious pages or imitate trusted services.

The Prefix/Suffix feature was another strong indicator because attackers often add symbols such as hyphens or additional text to create domains that look similar to legitimate websites.

In contrast, features such as Right_Click and HTTPS_Domain showed very low importance. This suggests that these features provide limited additional information for the classification process.

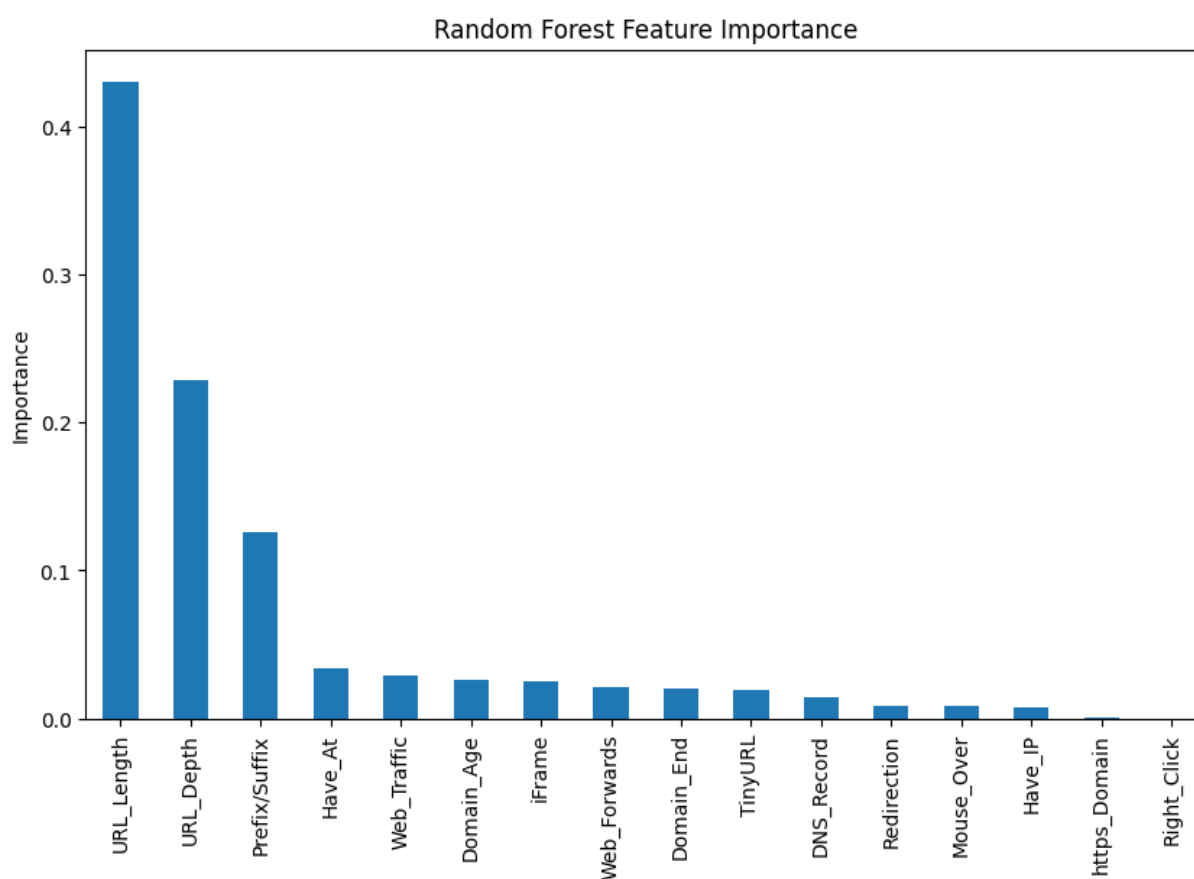


Figure 1: Random Forest feature importance analysis.

4.4 Feature Redundancy and Possible Improvements

Some redundancy may exist between features because several extracted characteristics describe related URL properties. For example, URL length and URL depth may be correlated because longer URLs often contain more directory levels.

Feature redundancy can be detected using correlation analysis or feature importance analysis. Highly correlated features may provide similar information and can sometimes be removed to simplify the model without significantly reducing performance.

Possible improvements include adding more advanced features such as domain reputation scores, website content analysis, SSL certificate information, and temporal features that capture changes in phishing behavior over time.

Additional feature selection techniques or dimensionality reduction methods could also be investigated to improve efficiency and reduce unnecessary complexity.

5. Reproducibility Analysis

Reproducibility is an important requirement in data science because experiments should be repeatable by other researchers. A cybersecurity model should not only achieve good results, but the training process, preprocessing steps, and evaluation methodology should also be clearly documented.

The original project provided access to the implementation code and dataset through a GitHub repository, which made it possible to reproduce the phishing detection experiment. The dataset file was available, and the main machine learning workflow could be recreated using common Python data science libraries.

5.1 Execution Environment

The reproduction experiment was implemented using Google Colab with Python. The main libraries used were:

- pandas and NumPy for data loading and processing.
- matplotlib and seaborn for visualization and exploratory data analysis.
- scikit-learn for preprocessing, model training, and evaluation.
- XGBoost for gradient boosting classification.

A fixed random seed (`random_state=42`) was used during model training and data splitting to make the experimental results reproducible.

5.2 Reproducibility Issues and Improvements

During reproduction, some improvements were introduced to make the workflow clearer and more reliable.

One issue identified was that the Domain column contained textual website identifiers. Since the selected machine learning models require numerical inputs, this column was removed before feature scaling and model training.

The reproduction also extended the original evaluation process by adding additional metrics such as Precision, Recall, F1-score, ROC-AUC, MCC, and confusion matrices. This provides a more complete analysis of model behavior compared with relying mainly on accuracy.

Overall, the original project was reproducible because the dataset and implementation were publicly available. However, adding clearer preprocessing documentation and a more detailed evaluation methodology improves transparency and makes the experiment easier to validate.

6. Experimental Results

The reproduction study evaluated multiple machine learning models for phishing website detection. The goal of the experiments was to compare different classification approaches and determine which model provides the best performance for distinguishing phishing websites from legitimate websites.

The dataset was divided into training and testing sets using an 80/20 split. Stratified splitting was applied to preserve the original class distribution between phishing and legitimate websites.

Four machine learning models were trained and evaluated:

- Logistic Regression
- Decision Tree
- Random Forest
- XGBoost

The experiments extended the original analysis by evaluating the models using several performance metrics instead of relying only on accuracy.

6.1 Evaluation Metrics

Several evaluation metrics were selected because phishing detection requires understanding different types of classification behavior.

Accuracy measures the percentage of correctly classified websites. Although useful, accuracy alone can be misleading in cybersecurity problems, especially when datasets are imbalanced.

Precision measures how many websites predicted as phishing are actually phishing. High precision reduces the number of legitimate websites incorrectly blocked.

Recall measures the ability of the model to identify phishing websites. Recall is especially important in phishing detection because a False Negative allows a malicious website to remain undetected.

The F1-score combines Precision and Recall into a single metric, providing a balance between the two.

ROC-AUC measures the ability of the model to separate phishing and legitimate classes across different classification thresholds.

Matthews Correlation Coefficient (MCC) was also used because it provides a balanced evaluation by considering True Positives, True Negatives, False Positives, and False Negatives.

6.2 Model Comparison Results

The experimental results showed that ensemble-based models achieved stronger performance compared with the baseline Logistic Regression model.

Logistic Regression provided a useful baseline but had lower Recall, meaning it missed more phishing websites compared with the other approaches.

Decision Tree improved the classification performance by capturing non-linear relationships between features.

Random Forest achieved the best overall balance between Precision, Recall, and F1-score by combining multiple decision trees and reducing overfitting.

XGBoost also achieved strong performance, demonstrating the effectiveness of gradient boosting methods for structured cybersecurity datasets.

6.3 Results Analysis

The model comparison results demonstrated that more advanced ensemble methods achieved better performance than the baseline model.

The Random Forest model achieved the strongest overall performance because it provided the best balance between detecting phishing websites and avoiding incorrect classifications. This improvement occurs because Random Forest combines predictions from multiple decision trees, which reduces overfitting and improves generalization.

XGBoost also achieved competitive results due to its gradient boosting approach, where models are trained sequentially to correct previous errors.

The results indicate that phishing detection benefits from models capable of capturing complex and non-linear relationships between URL and domain-based features.

	Model	Accuracy	Precision	Recall	F1 Score	MCC
0	Logistic Regression	0.7965	0.922967	0.647	0.760729	0.621428
1	Decision Tree	0.8650	0.925408	0.794	0.854682	0.737473
2	Random Forest	0.8670	0.924769	0.799	0.857296	0.740884
3	XGBoost	0.8625	0.971391	0.747	0.844545	0.745154

Figure 2: Performance comparison of the trained machine learning models.

6.4 ROC Curve Analysis

ROC curve analysis was performed to compare the ability of the models to distinguish between phishing and legitimate websites.

A higher ROC-AUC value indicates that a model is better at separating the two classes across different decision thresholds.

The ROC analysis confirmed that the ensemble-based approaches, particularly Random Forest and XGBoost, provided stronger discrimination capability compared with simpler models.

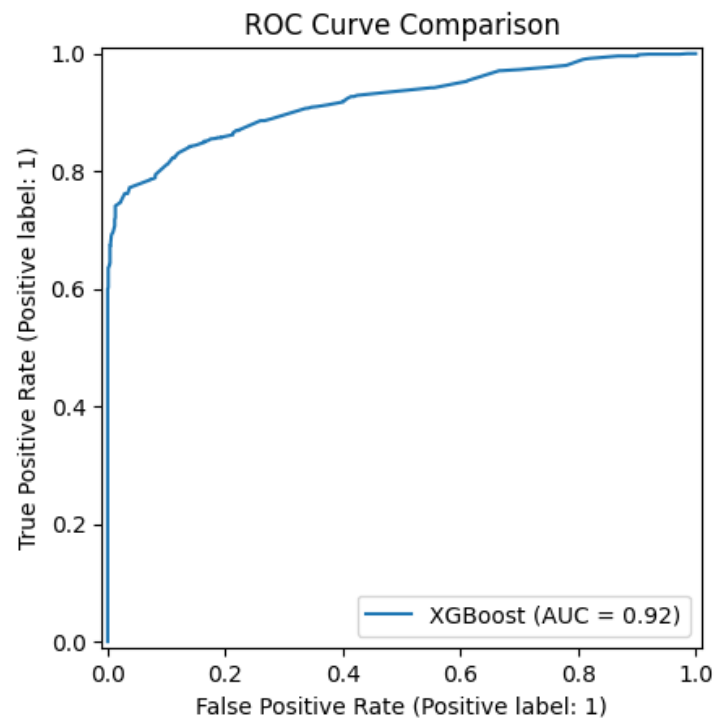


Figure 3: ROC curve comparison between classification models.

6.5 Error Analysis

The confusion matrices were analyzed to understand the types of mistakes made by the models.

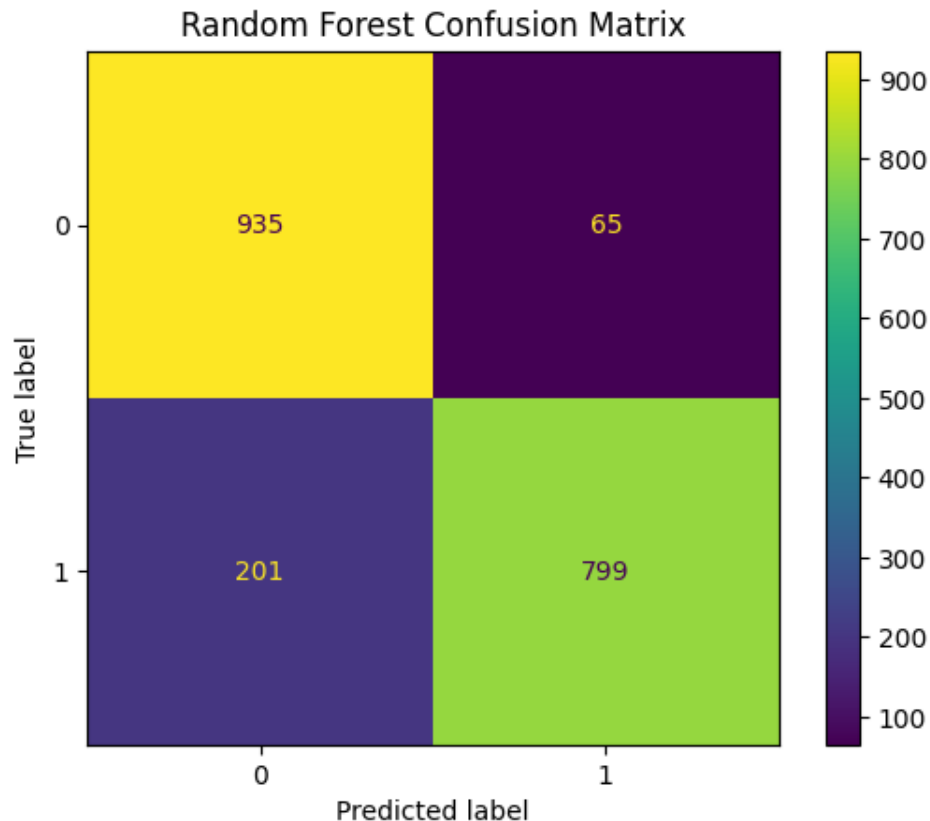


Figure 4: Random Forest confusion matrix analysis.

Two main error types exist:

False Positives:

A legitimate website is incorrectly classified as phishing. This may reduce user experience because safe websites could be unnecessarily blocked.

False Negatives:

A phishing website is incorrectly classified as legitimate. In cybersecurity, this type of error is usually more dangerous because users may interact with malicious websites without receiving a warning.

For phishing detection systems, minimizing False Negatives is especially important because the cost of allowing an attack is typically higher than incorrectly warning users about a safe website.

Therefore, Recall and F1-score were considered important metrics when evaluating the models, rather than focusing only on Accuracy.

7. Conclusions

This project reproduced and critically evaluated a machine learning approach for phishing website detection. The original project demonstrated that URL and domain-based features can be used to distinguish phishing websites from legitimate websites.

The reproduction experiments confirmed that machine learning models can achieve strong performance on this task. Among the evaluated algorithms, ensemble-based models such as Random Forest and XGBoost achieved the best results because they were able to capture complex relationships between the extracted features.

The main strength of the original approach is that it provides an automated and scalable method for detecting phishing websites without relying only on manually maintained blacklists. The feature-based approach allows models to identify suspicious patterns that may indicate malicious behavior.

However, several limitations were identified. The dataset used in the experiment is balanced, which may not fully represent real-world cybersecurity environments where phishing websites are much less frequent than legitimate websites. In addition, phishing strategies change continuously, meaning that models require regular updates using newer data.

Future improvements could include testing the models on larger and more recent datasets, adding real-time website features, applying additional feature selection techniques, and evaluating the system under realistic class imbalance conditions.

Overall, the reproduced results support the author's main claim that machine learning is effective for phishing detection. However, additional validation is required before deploying such systems in real-world cybersecurity applications.

References

[1] S. Gopal, "Phishing Website Detection by Machine Learning Techniques," GitHub Repository.

Available: <https://github.com/shreyagopal/Phishing-Website-Detection-by-Machine-Learning-Techniques>

[2] Scikit-learn Developers, "Scikit-learn: Machine Learning in Python Documentation."

Available: <https://scikit-learn.org/>

[3] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016.